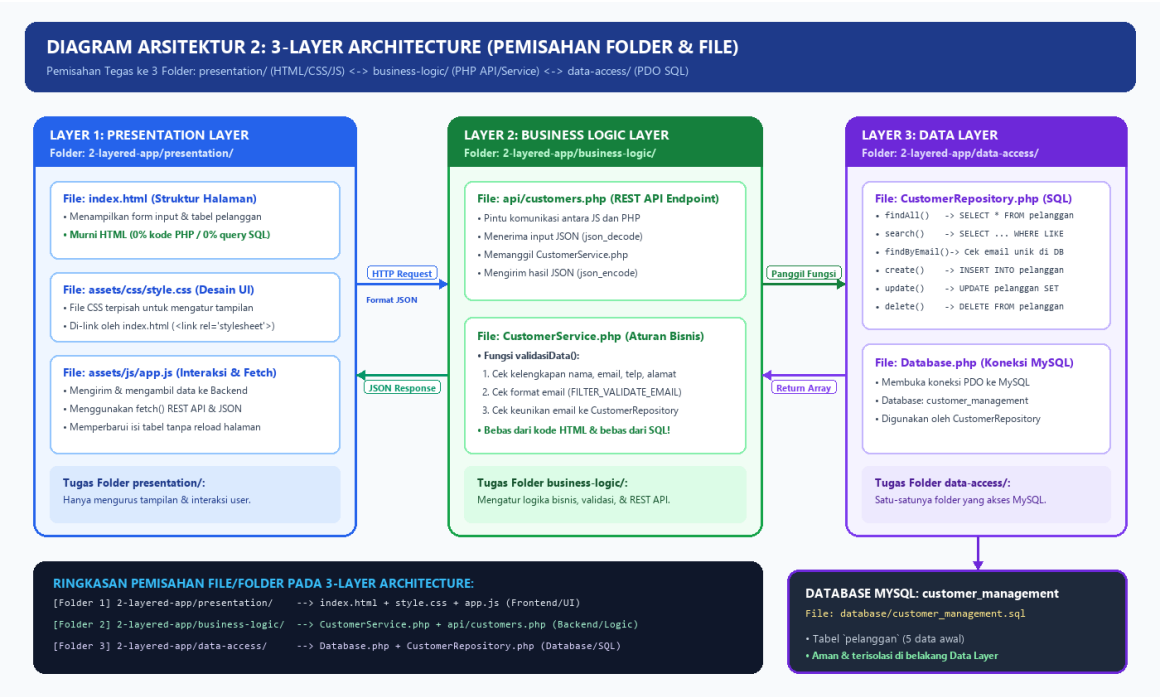
LAPORAN TUGAS KELOMPOK PEMROGRAMAN WEB

ANALISIS DAN IMPLEMENTASI PERBANDINGAN LAYERED VS NON-LAYERED WEB APPLICATION ARCHITECTURE

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)



Judul Tugas	Analisis dan Implementasi Perbandingan Layered vs Non-Layered Web Application Architecture
Studi Kasus	Customer Management System (Pengelolaan Data Pelanggan Perusahaan)
6 Fitur Wajib	1. Tambah Pelanggan, 2. Tampil Pelanggan, 3. Edit Pelanggan, 4. Hapus Pelanggan, 5. Pencarian (Nama/Email), 6. Validasi (Kelengkapan & Email Unik)
Aplikasi 1 (Non-Layered)	Arsitektur Monolitik 1 File (1-non-layered-app/index.php) — HTML, CSS, Validasi PHP, & Query SQL Menyatu
Aplikasi 2 (3-Layer)	Pemisahan 3 Folder: presentation/ (HTML/CSS/JS), business-logic/ (Service & REST API JSON), data-access/ (PDO MySQL)
Database SQL	database/customer_management.sql (Database MySQL customer_management, Tabel pelanggan + 5 Data Awal)

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, cara pengembang menyusun dan mengorganisasikan kode program (arsitektur perangkat lunak) sangat menentukan kemudahan pemeliharaan, kecepatan pencarian kesalahan (*debugging*), keamanan database, serta kemampuan aplikasi untuk dikembangkan secara berkelompok (kerja tim). Pada tahap awal belajar pemrograman web menggunakan PHP Native dan MySQL, pendekatan yang paling sering digunakan adalah menempatkan seluruh kode tampilan (HTML dan CSS), logika proses bisnis (validasi PHP), serta koneksi dan perintah query database (SQL) di dalam satu file yang sama. Pendekatan ini dikenal sebagai **Non-Layered Architecture** atau **Arsitektur Monolitik Sederhana**.

Meskipun pendekatan satu file tersebut terasa cepat dan mudah ketika membuat program yang sangat kecil, masalah serius akan muncul ketika fitur aplikasi bertambah kompleks atau ketika aplikasi harus dikerjakan bersama oleh beberapa anggota tim. Kode program menjadi tercampur aduk (*spaghetti code*), perubahan desain tampilan rawan merusak query database, dan penelusuran sumber kesalahan (*bug*) menjadi sulit. Untuk mengatasi permasalahan tersebut, digunakan konsep **3-Layer Web Application Architecture** yang membagi sistem secara tegas menjadi tiga lapisan tanggung jawab, yaitu **Presentation Layer**, **Application / Business Logic Layer**, dan **Data Layer**.

1.2 Rumusan Masalah

1. Apa pengertian, karakteristik, serta perbedaan mendasar antara **Non-Layered Architecture** dan **3-Layer Web Application Architecture**?
2. Bagaimana cara mengimplementasikan studi kasus **Customer Management System** (dengan 6 fitur wajib) menggunakan pendekatan Non-Layered (1 file PHP) dibandingkan dengan pendekatan 3-Layer (3 folder terpisah dengan komunikasi REST API & JSON)?
3. Bagaimana perbandingan alur proses *request-response* serta struktur pemisahan file/folder pada kedua arsitektur tersebut?
4. Berdasarkan hasil uji coba pada berbagai unsur (struktur file/folder, kecepatan pembuatan awal, pengerjaan individu vs tim, kemudahan penelusuran kesalahan/bug, kecepatan respons UI, keamanan edit desain, penambahan fitur, dan reusabilitas API), manakah arsitektur yang lebih unggul beserta alasannya?

1.3 Tujuan Penulisan & Implementasi

- Memahami konsep dasar dan fungsi masing-masing lapisan pada **3-Layer Web Application Architecture**.
- Membangun dua aplikasi web **Customer Management System** menggunakan PHP Native dan MySQL dengan fitur yang identik tetapi arsitektur yang berbeda (Non-Layered vs 3-Layer).
- Menyajikan perbandingan visual melalui **Diagram Arsitektur (Pemisahan File/Folder)**, **Screenshot Code** yang mewakili konsep masing-masing arsitektur, serta **Screenshot Tampilan Web**.
- Menyusun **Satu Tabel Utama Hasil Pengujian dan Perbandingan** untuk menentukan keunggulan serta alasan pemilihan masing-masing arsitektur.

BAB 2. LANDASAN TEORI & PENGERTIAN ARSITEKTUR APLIKASI WEB

2.1 Pengertian 3-Layer Web Application Architecture (Halaman 2–3 PDF)

3-Layer Web Application Architecture adalah konsep pengembangan aplikasi web dengan membagi sistem menjadi tiga lapisan (*layer*) yang memiliki tugas dan tanggung jawab berbeda. Tujuan utamanya adalah membuat aplikasi lebih **terstruktur**, **mudah dikembangkan**, **mudah dipelihara**, **aman**, dan **fleksibel**. Ketiga lapisan tersebut adalah:

1. **Presentation Layer** (Lapisan Antarmuka / Frontend)
2. **Application / Business Logic Layer** (Lapisan Logika Bisnis / Backend)

3. **Data Layer** (Lapisan Pengelolaan Data / Database)

2.2 Penjelasan Detail Ketiga Lapisan pada 3-Layer Architecture (Halaman 4–11 PDF)

A. Presentation Layer (Frontend): Merupakan bagian aplikasi yang berinteraksi langsung dengan pengguna. Lapisan ini bertugas menampilkan informasi (seperti tabel pelanggan dan pesan notifikasi) serta menerima masukan dari pengguna melalui formulir. Teknologi yang digunakan meliputi **HTML** (membangun struktur halaman), **CSS** (mengatur tampilan dan desain), dan **JavaScript** (membuat halaman interaktif serta mengirim/menerima data ke backend melalui API). *Intinya: Presentation Layer bertanggung jawab terhadap apa yang dilihat dan digunakan oleh pengguna.*

B. Application / Business Logic Layer (Backend): Merupakan bagian yang menangani proses bisnis atau aturan utama aplikasi. Lapisan ini menerima permintaan (*request*) dari frontend, memproses dan memvalidasinya (misalnya memeriksa apakah seluruh kolom sudah terisi dan apakah email pelanggan bersifat unik), kemudian berkomunikasi dengan Data Layer jika diperlukan. Teknologi yang digunakan pada proyek ini adalah **PHP Native** (tanpa framework) yang menyediakan **REST API berformat JSON**. *Intinya: Business Logic Layer bertanggung jawab terhadap bagaimana aplikasi bekerja dan mengambil keputusan.*

C. Data Layer (Database): Bertugas menyimpan, mengambil, memperbarui, dan menghapus data. Lapisan ini berisi database (**MySQL**) beserta mekanisme akses data (koneksi PDO dan *prepared statement* SQL di dalam kelas Repository). *Intinya: Data Layer bertanggung jawab terhadap penyimpanan dan pengelolaan data aplikasi.*

Aspek	Presentation Layer	Business Logic Layer	Data Layer
Fungsi Utama	Tampilan & Interaksi User	Proses & Aturan Bisnis	Penyimpanan & Query Data
Berinteraksi Dengan	Pengguna (Browser)	Frontend & Data Layer	Business Logic Layer (Backend)
Teknologi Proyek Ini	HTML5, CSS3, JavaScript (Fetch)	PHP Native (Service & REST API)	PDO PHP & Database MySQL
Folder pada Proyek	2-layered-app/presentation/	2-layered-app/business-logic/	2-layered-app/data-access/
Tanggung Jawab	Menampilkan & menerima data	Memvalidasi & mengolah data	Menyimpan & mengambil data SQL

Tabel 2.1 Ringkasan Tanggung Jawab Ketiga Layer (Sesuai Halaman 11 PDF)

2.3 Lima Alasan Utama Menggunakan Arsitektur 3-Layer (Halaman 12 PDF)

1. **Separation of Concerns:** Setiap layer mempunyai tanggung jawab masing-masing yang terpisah secara tegas sehingga kode tidak tercampur.
2. **Maintainability (Kemudahan Pemeliharaan):** Perubahan pada tampilan (HTML/CSS) tidak harus mengubah struktur database maupun logika bisnis.
3. **Scalability (Kemudahan Pengembangan):** Setiap layer dapat dikembangkan atau diperbesar kapasitasnya sesuai kebutuhan tanpa merusak layer lain.
4. **Security (Keamanan):** Database tidak pernah diakses secara langsung dari halaman antarmuka pengguna, melainkan terlindungi di balik Business Logic Layer dan Data Layer.
5. **Reusability (Penggunaan Ulang):** Backend REST API yang menghasilkan format JSON dapat digunakan secara bersamaan oleh aplikasi Web maupun aplikasi Mobile (Android/iOS).

2.4 Pengertian & Cara Kerja Aplikasi Tanpa 3-Layer / Non-Layered (Halaman 14–16 PDF)

Aplikasi Tanpa 3-Layer (Non-Layered Architecture) biasa disebut sebagai **arsitektur monolitik sederhana** (*monolithic application*), yaitu pendekatan pengembangan aplikasi di mana kode frontend (tampilan), proses bisnis (validasi), dan akses database (query SQL) tidak dipisahkan secara tegas. Pada pendekatan ini, seluruh proses ditempatkan di dalam **satu file PHP (index.php)** mulai dari: (1) Menampilkan formulir HTML, (2) Menerima input pengguna, (3) Memvalidasi data, (4) Mengakses database MySQL, hingga (5) Menampilkan hasil.

Catatan Penting (Halaman 16 PDF): *Aplikasi satu file ini bukan berarti sama sekali tidak memiliki tiga tanggung jawab tersebut. Fungsi tampilan, proses bisnis, dan akses data tetap ada di dalamnya, tetapi tidak dipisahkan menjadi layer/folder yang jelas.*

Aspek	Tanpa 3-Layer (Non-Layered)	Dengan 3-Layer Architecture
Struktur	Sederhana (menyatu)	Terpisah secara tegas dalam 3 layer
File Program	Bisa hanya satu file (index.php)	Terdiri dari beberapa komponen/folder terpisah
Frontend	Menyatu langsung dengan kode PHP	Terpisah di folder presentation/
Business Logic	Menyatu dengan kode tampilan aplikasi	Dipisahkan di folder business-logic/
Database Access	Langsung ditulis di dalam file PHP utama	Melalui komponen khusus di folder data-access/
Pemeliharaan	Lebih sulit jika aplikasi mulai kompleks	Lebih terstruktur dan aman dimodifikasi
Pengembangan	Cepat untuk aplikasi kecil / individu	Lebih terorganisasi untuk tim & jangka panjang
Pengujian & Debugging	Cenderung lebih sulit menelusuri error	Lebih mudah karena error terisolasi per layer

Tabel 2.2 Perbandingan Konsep Dasar Tanpa 3-Layer vs Dengan 3-Layer (Sesuai Halaman 16 PDF)

BAB 3. STUDI KASUS: CUSTOMER MANAGEMENT SYSTEM & DATABASE

3.1 Spesifikasi 6 Fitur Wajib (Halaman 18–19 PDF)

Sesuai instruksi pada Halaman 18–19 PDF, studi kasus yang dibangun adalah **Customer Management System**, yaitu aplikasi web untuk mengelola data pelanggan sebuah perusahaan. Kedua aplikasi (baik Non-Layered maupun 3-Layer) wajib mengimplementasikan **6 fitur utama** berikut:

No	Nama Fitur Wajib	Keterangan Fungsi	Implementasi pada Proyek
1	**Tambah Pelanggan**	Memasukkan data pelanggan baru	Input nama, email, telepon, dan alamat ke tabel pelanggan
2	**Tampil Pelanggan**	Menampilkan seluruh data pelanggan	Menampilkan tabel daftar pelanggan diurutkan dari ID terbaru
3	**Edit Pelanggan**	Mengubah informasi pelanggan	Mengambil data berdasarkan ID lalu memperbarui baris di MySQL
4	**Hapus Pelanggan**	Menghapus data pelanggan	Menghapus data pelanggan berdasarkan ID dengan konfirmasi
5	**Pencarian**	Mencari berdasarkan nama atau email	Query pencarian menggunakan WHERE nama LIKE ? OR email LIKE ?
6	**Validasi**	Memeriksa kelengkapan & keunikan email	Menolak kolom kosong, format email salah, atau email yang sudah terdaftar

Tabel 3.1 Enam Fitur Wajib Studi Kasus Customer Management System (Sesuai Halaman 19 PDF)

3.2 Struktur Tabel Database MySQL (File: database/customer_management.sql)

Kedua aplikasi menggunakan database MySQL bernama customer_management dengan tabel pelanggan yang sudah dilengkapi 5 data pelanggan awal (*seed data*):

Nama Kolom	Tipe Data	Constraint / Indeks	Keterangan
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	ID unik setiap pelanggan
nama	VARCHAR(100)	NOT NULL, INDEX	Nama lengkap pelanggan perusahaan
email	VARCHAR(100)	NOT NULL, UNIQUE	Alamat email pelanggan (wajib unik / tidak boleh sama)
telepon	VARCHAR(25)	NOT NULL	Nomor telepon aktif pelanggan
alamat	TEXT	NOT NULL	Alamat lengkap pelanggan
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu pencatatan data pelanggan

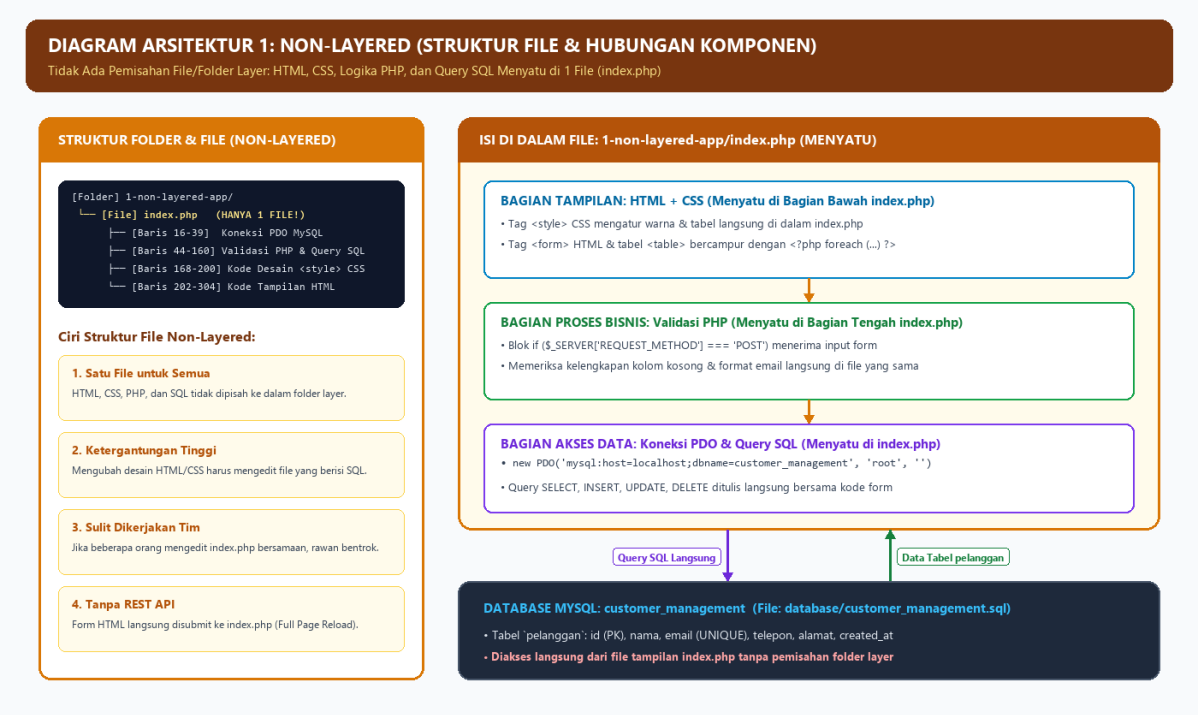
Tabel 3.2 Struktur Tabel pelanggan pada File database/customer_management.sql

3.3 Ketentuan Teknis Kedua Aplikasi (Halaman 20–21 PDF)

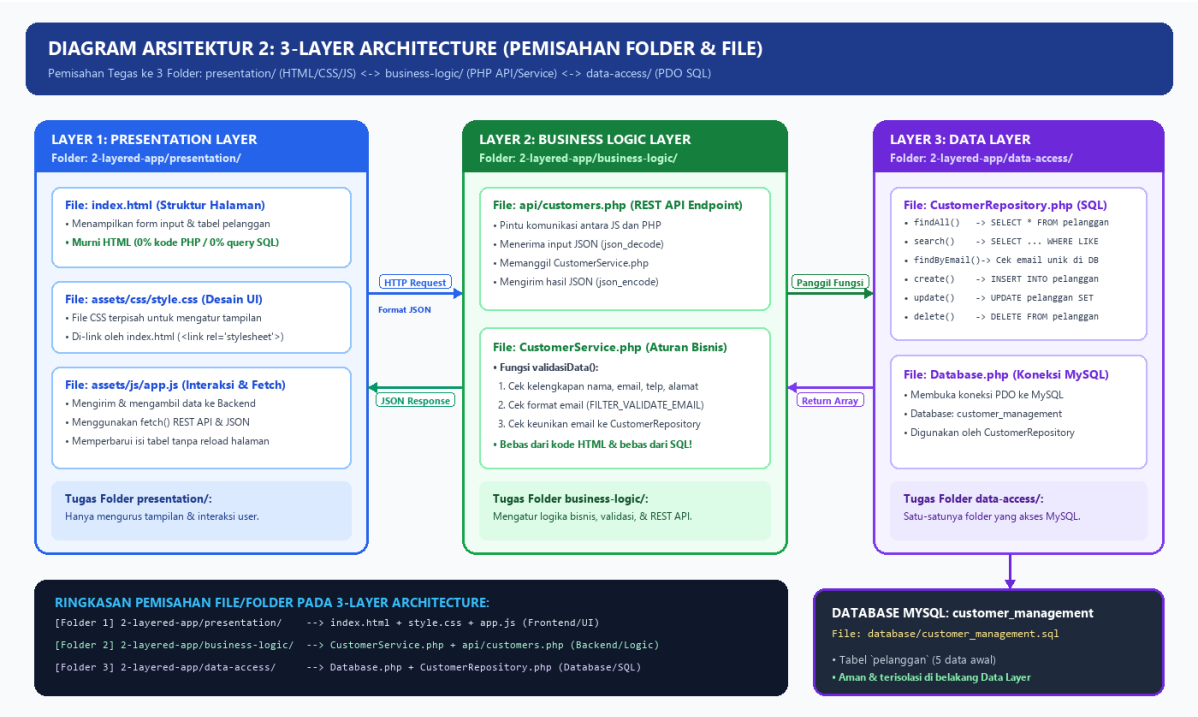
- **Aplikasi Pertama (Non-Layered Architecture — Halaman 20 PDF):** Dibangun menggunakan PHP Native dan MySQL di mana tampilan HTML digabungkan dengan PHP, proses bisnis ditempatkan dalam file yang sama dengan tampilan, koneksi database dan query ditulis langsung dalam file aplikasi (1-non-layered-app/index.php), tidak menggunakan REST API, dan seluruh fitur CRUD berjalan.
- **Aplikasi Kedua (3-Layer Architecture — Halaman 21 PDF):** Membangun kembali aplikasi yang sama menggunakan PHP Native (non-framework) dengan memisahkan tanggung jawab menjadi 3 folder: **Presentation Layer** (presentation/) menangani tampilan & interaksi user, **Business Logic Layer** (business-logic/) menangani proses & aturan bisnis, serta **Data Layer** (data-access/) menangani komunikasi dengan database MySQL. Komunikasi antara frontend dan backend wajib menggunakan **REST API dan JSON**.

BAB 4. DIAGRAM ARSITEKTUR (PEMISAHAN FILE/FOLDER & HUBUNGAN KOMPONEN)

Sesuai arahan dosen, diagram arsitektur dirancang untuk memperlihatkan secara jelas **struktur pemisahan file/folder** serta bagaimana komponen **HTML, CSS, JavaScript, PHP, dan SQL** saling berhubungan pada kedua pendekatan:



Gambar 4.1 Diagram Arsitektur Non-Layered: Seluruh Kode (HTML, CSS, Validasi PHP, & Query SQL) Menyatu dalam 1 File index.php



Gambar 4.2 Diagram Arsitektur 3-Layer: Pemisahan Tegas ke dalam 3 Folder Layer (presentation/, business-logic/, data-access/)

BAB 5. SCREENSHOT CODE KEDUA KONSEP ARSITEKTUR

Untuk membuktikan perbedaan konsep secara nyata pada level kode sumber (*source code*), berikut adalah perbandingan langsung potongan kode utama pada aplikasi **Non-Layered** (1 file monolitik) dan **3-Layer Architecture** (3 folder terpisah):

SCREENSHOT CODE 1: KONSEP NON-LAYERED ARCHITECTURE (1 FILE MONOLITIK)
File: 1-non-layered-app/index.php — Koneksi Database. Validasi Proses Bisnis. Query SQL, CSS, dan HTML Menyatu dalam 1 File

1-non-layered-app/index.php (Seluruh Kode Menyatu dalam 1 File) — NON-LAYERED (MONOLITIK)

```php
// ============================================================================
// BAGIAN 1: KONEKSI LANGSUNG KE DATABASE MYSQL (Di dalam index.php)
// ============================================================================
$koneksi = new PDO("mysql:host=localhost;dbname=customer_management", "root", "");

// ============================================================================
// BAGIAN 2: PROSES BISNIS (VALIDASI) & QUERY CRUD (Menyatu di index.php)
// ============================================================================
if (isset($_GET['hapus'])) {
    $stmt = $koneksi->prepare("DELETE FROM pelanggan WHERE id = ?");
    $stmt->execute([(int) $_GET['hapus']]);
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // A. Validasi Kelengkapan Data
    if ($nama === '' || $email === '' || $telepon === '' || $alamat === '') {
        $pesan_error[] = "Semua kolom pelanggan wajib diisi.";
    }
    // B. Validasi Keunikan Email (Langsung query SQL di file tampilan)
    $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ?");
    $stmtCek->execute([$email]);
    if ($stmtCek->fetch()) {
        $pesan_error[] = "Email sudah terdaftar di database!";
    }
    // C. Eksekusi Query INSERT / UPDATE Langsung
    $stmt = $koneksi->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?,?,?,?)");
    $stmt->execute([$nama, $email, $telepon, $alamat]);
}
// Query Tampil & Pencarian Pelanggan
$daftar_pelanggan = $koneksi->query("SELECT * FROM pelanggan ORDER BY id DESC")->fetchAll();
?>
<!-- ============================================================================
     BAGIAN 3: TAMPILAN HTML & CSS (Menyatu langsung di bawah kode PHP)
     ============================================================================ -->
<!DOCTYPE html><html lang="id"><head><style> /* Kode CSS di sini */ </style></head>
```

Gambar 5.1 Screenshot Code Non-Layered Architecture (1-non-layered-app/index.php): Koneksi PDO, Validasi, Query SQL, CSS, dan HTML Menyatu

SCREENSHOT CODE 2: KONSEP 3-LAYER ARCHITECTURE (TERPISAH 3 FOLDER LAYER)
Pemisahan Tegas: Presentation Layer (app.js) <-> Business Logic Layer (CustomerService.php & api) <-> Data Layer (CustomerRepository.php)

presentation/app.js — LAYER 1: PRESENTATION

```javascript
// Folder: 2-layered-app/presentation/
// Murni Frontend (Tanpa PHP & Tanpa SQL)
const URL_API = '../business-logic/api/customers.php';

// 1. Ambil Data via GET REST API (JSON)
async function muatDataPelanggan(kataKunci = '') {
  const response = await fetch(url);
  const hasil = await response.json();
  tbody.innerHTML = hasil.data.map(...);
}

// 2. Kirim Data via POST/PUT JSON
const response = await fetch(url, {
  method: idEdit ? 'PUT' : 'POST',
  headers: {'Content-Type':'application/json'},
  body: JSON.stringify(dataKirim)
});
const hasil = await response.json();
```

CustomerService.php — LAYER 2: BUSINESS LOGIC

```php
// Folder: 2-layered-app/business-logic/
// Aturan Bisnis, Validasi, & REST API JSON
class CustomerService {
  public function createCustomer($input) {
    $data = $this->rapikanInput($input);
    $errors = $this->validasiData($data);
    if (!empty($errors)) {
      return ['status_code' => 422,
              'success' => false,
              'errors' => $errors];
    }
    $baru = $this->repo->create($data);
    return ['status_code' => 201,
            'success' => true,
            'data' => $baru];
  }
  // Validasi kelengkapan & email unik
  private function validasiData($data) {...}
}
```

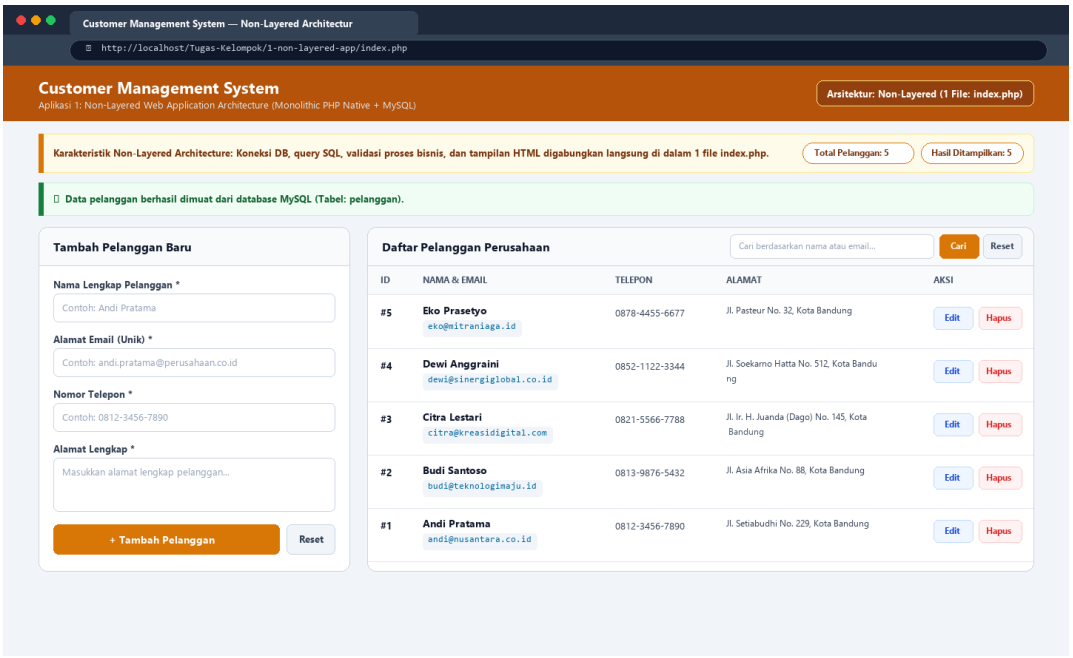
CustomerRepository.php — LAYER 3: DATA LAYER

```php
// Folder: 2-layered-app/data-access/
// Khusus Koneksi PDO & Query SQL MySQL
class CustomerRepository {
  public function findAll() {
    return $this->db->query(
      'SELECT * FROM pelanggan ORDER BY id DESC'
    )->fetchAll();
  }
  public function findByEmail($email) {
    $stmt = $this->db->prepare(
      'SELECT * FROM pelanggan WHERE email = ?'
    );
  }
  public function create($data) {
    $stmt = $this->db->prepare(
      'INSERT INTO pelanggan VALUES (?,?,?,?)'
    );
    $stmt->execute([...]);
  }
}
```

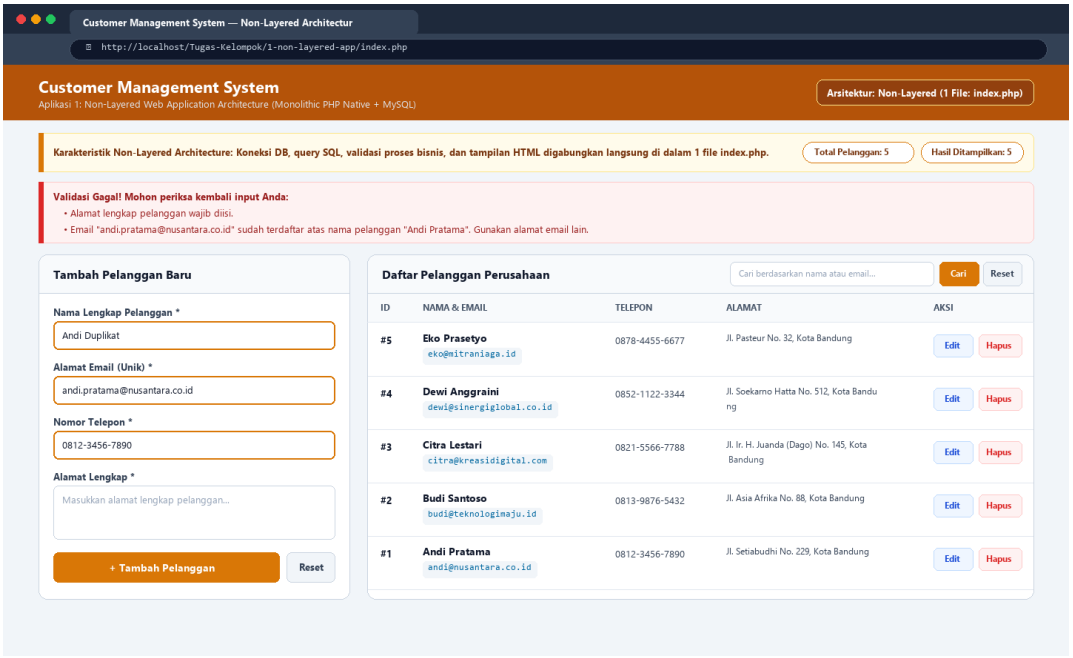
Gambar 5.2 Screenshot Code 3-Layer Architecture: Pemisahan Tegas antara app.js (Presentation), CustomerService.php (Logic), dan CustomerRepository.php (Data)

BAB 6. SCREENSHOT TAMPILAN WEB & PERBANDINGAN ALUR PROSES

6.1 Screenshot Tampilan Aplikasi 1 (Non-Layered Architecture)

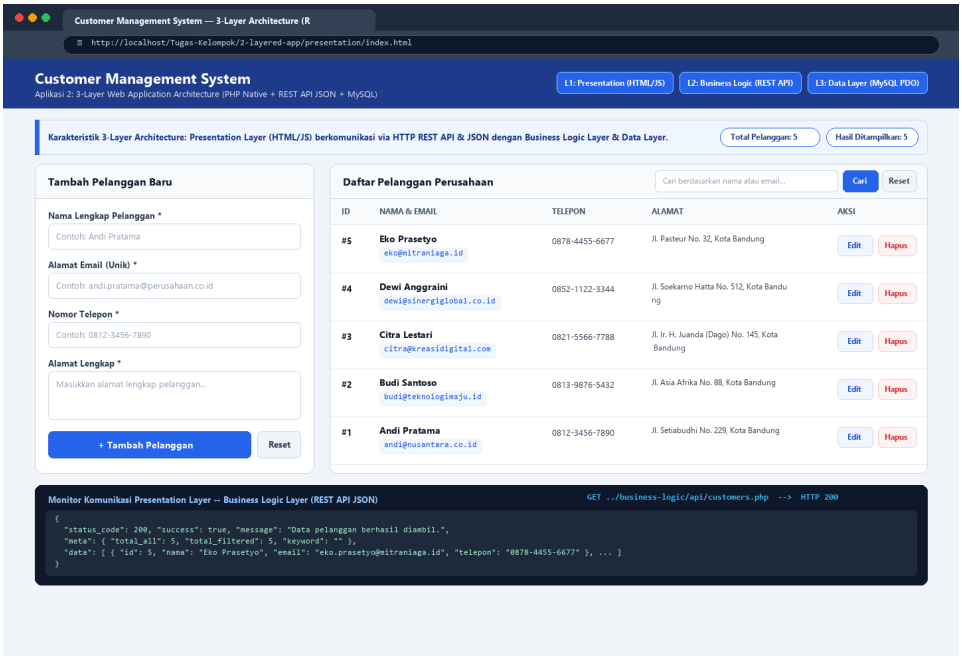


Gambar 6.1 Tampilan Utama & Tabel Daftar Pelanggan pada Aplikasi 1 (1-non-layered-app/index.php)

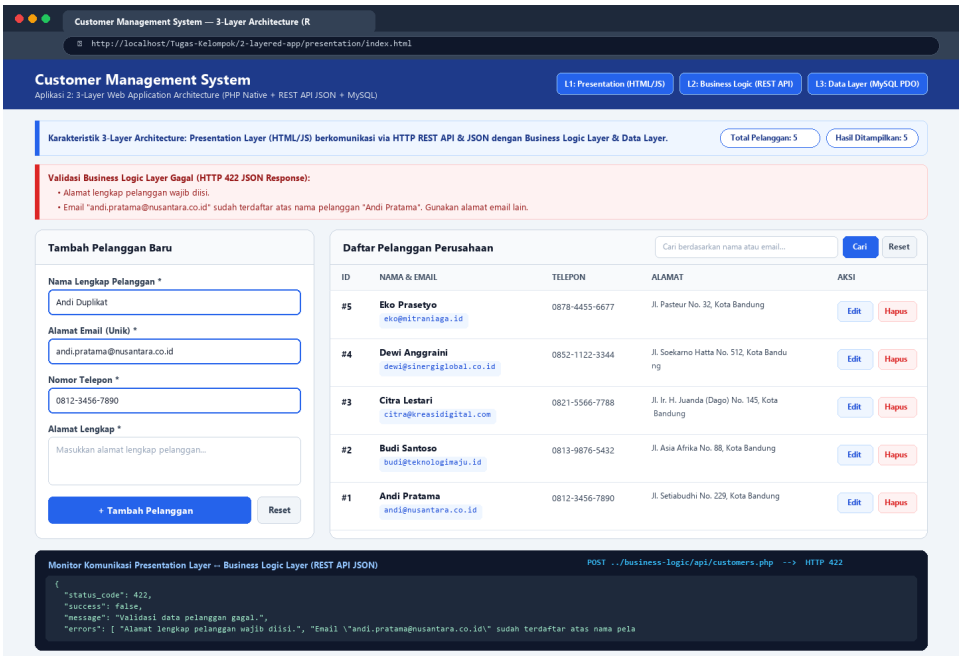


Gambar 6.2 Tampilan Pesan Validasi Kelengkapan Data & Email Unik pada Aplikasi 1

6.2 Screenshot Tampilan Aplikasi 2 (3-Layer Architecture dengan REST API & JSON)



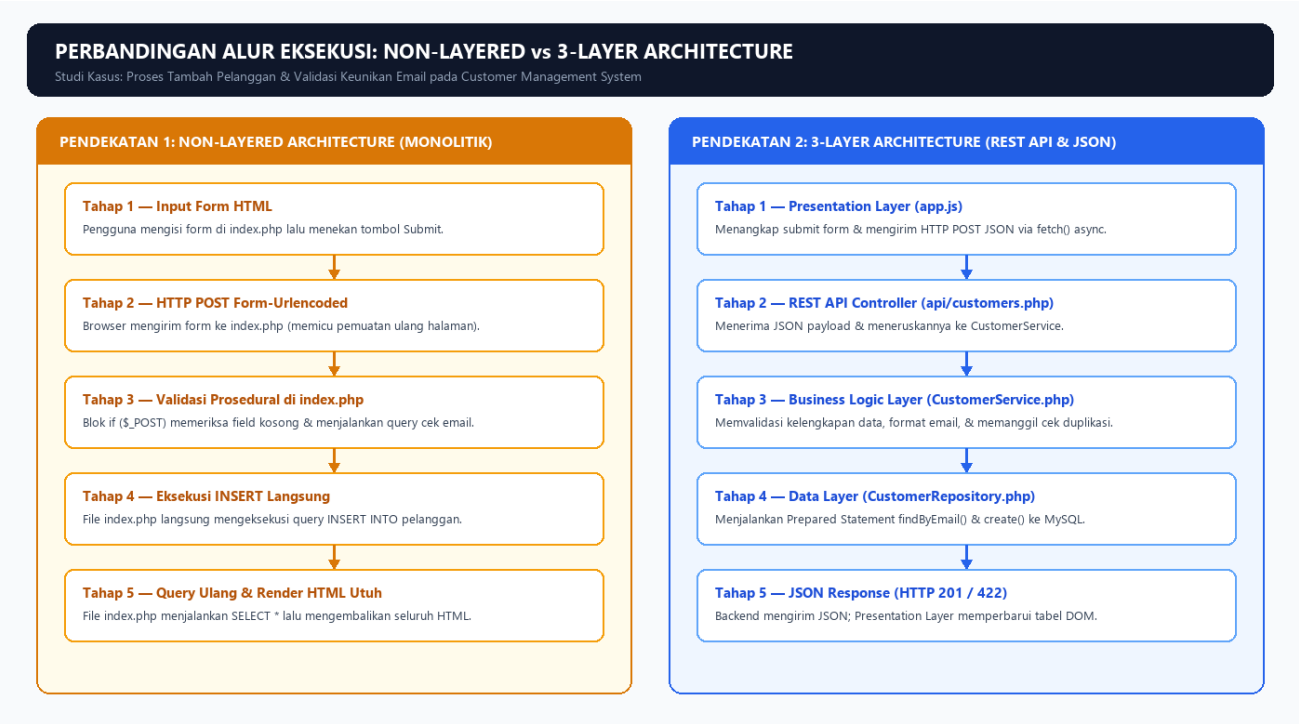
Gambar 6.3 Tampilan Utama Aplikasi 2 (3-Layer Architecture) beserta Panel Monitor Respons JSON (HTTP 200 OK)



Gambar 6.4 Tampilan Validasi Business Logic Layer (HTTP 422 Unprocessable Entity JSON) pada Aplikasi 2

6.3 Perbandingan Alur Proses Request & Response Kedua Aplikasi

Setelah melihat demo dan tampilan kedua aplikasi, perbedaan cara kerja di balik layar antara pengiriman form tradisional (*Full Page Reload*) pada Non-Layered dan komunikasi asinkron **REST API & JSON** pada 3-Layer dapat diringkas pada diagram berikut:



Gambar 6.5 Diagram Perbandingan Alur Proses: 6 Tahap Non-Layered (Full Page Reload) vs 7 Tahap 3-Layer (REST API & JSON)

BAB 7. TABEL HASIL PENGUJIAN DAN PERBANDINGAN AKHIR

Sesuai arahan dosen pengampu, berikut adalah **Satu Tabel Utama Hasil Pengujian dan Perbandingan** yang merangkum unsur-unsur apa saja yang diuji coba, bagaimana hasilnya pada pendekatan **Tanpa Layer (Non-Layered)** dibandingkan **Menggunakan 3-Layer**, siapa yang lebih unggul pada masing-masing unsur, beserta alasan lengkapnya:

No	Unsur yang Diuji Coba	Tanpa Layer (Non-Layered)	Menggunakan 3-Layer	Lebih Unggul	Alasan Mengapa Lebih Unggul
1	**Struktur Pemisahan File & Folder (Hubungan HTML, CSS, & PHP)**	Tidak dipisahkan. HTML, CSS, validasi PHP, dan query SQL digabung dalam 1 file (index.php).	Dipisahkan tegas ke 3 folder: presentation/ (HTML/CSS/JS), business-logic/ (PHP API), & data-access/ (SQL).	**3-Layer Architecture**	Kode jauh lebih rapi dan terorganisasi karena setiap file hanya memiliki satu tanggung jawab spesifik.
2	**Kecepatan Pembuatan Awal (Initial Development Speed)**	Sangat cepat dibuat di awal karena programmer cukup menulis langsung di 1 file tanpa membuat REST API.	Membutuhkan waktu awal lebih lama karena harus membuat kontrak JSON, Service, dan Repository di 3 folder.	**Non-Layered Architecture**	Untuk aplikasi yang sangat kecil/sederhana, menulis di 1 file lebih praktis tanpa overhead pembuatan banyak file.
3	**Pengerjaan secara Individu vs Tim (Teamwork & Kolaborasi)**	Cocok untuk 1 orang (individu), tetapi buruk untuk tim karena semua orang mengedit 1 file index.php yang sama.	Sangat ideal untuk kerja tim: anggota A mengerjakan folder presentation/, B di business-logic/, C di data-access/.	**3-Layer Architecture**	Pembagian tugas kelompok menjadi paralel dan tidak saling menimpa kode (bebas merge conflict di Git).
4	**Kemudahan Penelusuran Kesalahan (Debugging / Error Tracing)**	Sulit ditelusuri saat kode membesar karena harus mencari apakah error berasal dari HTML, validasi, atau SQL di 1 file.	Sangat mudah dilacak: error tampilan cek di presentation/, error validasi di CustomerService, error SQL di Repository.	**3-Layer Architecture**	Sumber bug langsung terisolasi per folder layer sehingga proses perbaikan kesalahan jauh lebih cepat.
5	**Kecepatan & Kenyamanan Interaksi (Pengalaman Pengguna / UX)**	Setiap kali tambah/cari/hapus data, browser harus memuat ulang (reload) seluruh halaman HTML dari awal.	Lebih cepat & responsif karena hanya mengirim/menerima data JSON via fetch() tanpa me-reload seluruh halaman.	**3-Layer Architecture**	Pertukaran data menggunakan REST API & JSON lebih ringan dibandingkan me-render ulang seluruh halaman HTML.
6	**Kemudahan Pemeliharaan Desain (Maintainability UI vs Database)**	Berisiko tinggi: saat mengubah desain tabel HTML/CSS, programmer bisa tidak sengaja merusak kode query SQL di atasnya.	Sangat aman: perubahan desain di index.html / style.css sama sekali tidak menyentuh file PHP maupun query SQL.	**3-Layer Architecture**	Perubahan pada tampilan antarmuka tidak memengaruhi logika bisnis maupun struktur akses database.
7	**Kemudahan Pengembangan Fitur (Scalability / Fitur Bertambah)**	Semakin banyak fitur ditambahkan, file index.php semakin panjang dan rumit (menjadi spaghetti code).	Sangat mudah dikembangkan: penambahan fitur baru cukup menambah fungsi secara modular di masing-masing layer.	**3-Layer Architecture**	Struktur modular menjaga kode tetap bersih meskipun fitur aplikasi terus bertambah besar.
8	**Keamanan & Penggunaan Ulang (Security & Reusability API)**	Query database berada di file tampilan dan tidak bisa dipakai oleh aplikasi Mobile (karena outputnya HTML).	Database terisolasi di Data Layer, dan endpoint REST API JSON (customers.php) bisa langsung dipakai Web & Mobile.	**3-Layer Architecture**	Satu Backend REST API JSON dapat melayani berbagai platform sekaligus (Web maupun Android/iOS).

Tabel 7.1 Tabel Utama Hasil Pengujian dan Perbandingan Non-Layered vs 3-Layer Architecture

BAB 8. JAWABAN 5 PERTANYAAN DISKUSI DAN KESIMPULAN

8.1 Jawaban Lengkap 5 Pertanyaan Diskusi (Halaman 17 PDF)

1. Mana yang lebih mudah dibuat?

Non-Layered Architecture lebih mudah dan cepat dibuat di awal (khususnya jika dikerjakan sendiri untuk aplikasi berskala kecil), karena programmer cukup menulis koneksi database, validasi PHP, query SQL, dan tampilan HTML di dalam satu file index.php tanpa perlu membuat kontrak REST API JSON maupun membagi kode ke dalam beberapa folder.

2. Mana yang lebih mudah diperbaiki ketika terjadi kesalahan (bug)?

3-Layer Architecture jauh lebih mudah diperbaiki dan ditelusuri. Karena adanya pemisahan folder per layer secara tegas, penelusuran kesalahan (*debugging*) langsung terarah: jika terjadi kesalahan tampilan maka diperiksa di folder presentation/, jika aturan validasi keliru diperiksa di CustomerService.php (folder business-logic/), dan jika query SQL bermasalah diperiksa di CustomerRepository.php (folder data-access/).

3. Mana yang lebih mudah dikembangkan ketika fitur bertambah?

3-Layer Architecture jauh lebih mudah dikembangkan. Penambahan fitur baru maupun pembagian tugas di dalam tim dapat dilakukan secara paralel per folder layer tanpa saling merusak kode anggota lain. Selain itu, endpoint REST API JSON (api/customers.php) dapat langsung digunakan ulang oleh aplikasi Mobile (Android/iOS) tanpa menulis ulang logika bisnis.

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Risiko utamanya adalah kode menjadi sangat panjang dan tercampur aduk (*spaghetti code*), perubahan sederhana pada desain HTML/CSS berisiko tinggi merusak logika validasi PHP atau query SQL di atasnya, sangat sulit dikerjakan bersama oleh tim karena rawan bentrok (*merge conflict*) pada satu file yang sama, serta sulit diuji secara terpisah.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk aplikasi yang sangat sederhana, skrip satu halaman, prototipe cepat (*MVP*), atau tugas individu berskala kecil, pendekatan tanpa layer justru praktis dan hemat waktu. Namun, untuk aplikasi perusahaan yang dikerjakan secara tim, membutuhkan pemeliharaan jangka panjang, dan terus berkembang, **3-Layer Architecture mutlak lebih unggul**.

8.2 Kesimpulan Akhir

Berdasarkan seluruh analisis teori dan hasil uji coba implementasi studi kasus **Customer Management System**, dapat disimpulkan bahwa **Non-Layered Architecture** hanya unggul dari segi kecepatan pembuatan awal untuk aplikasi kecil yang dikerjakan oleh satu orang. Sebaliknya, pada 7 dari 8 unsur pengujian lainnya—meliputi **kerapian pemisahan file/folder**, **kemudahan kerja sama tim**, **kemudahan penelusuran kesalahan (debugging)**, **kecepatan respons antarmuka tanpa reload**, **keamanan modifikasi desain**, **kemudahan penambahan fitur baru**, serta **reusabilitas REST API JSON**—pendekatan **3-Layer Web Application Architecture** terbukti jauh lebih unggul.